

PORTADA

Análisis y Diseño de Sistemas II

Semana 8 — SOLID: Principio de Responsabilidad Única (SRP)

Universidad de Antioquía · Ingeniería de Sistemas



APERTURA

**Una clase que valida, guarda, notifica
y audita — y nadie sabe qué se rompe si la tocan**

TRANSICIÓN

De la intuición al principio formal

La semana pasada, en cohesión y acoplamiento, propusimos una pregunta simple para detectar clases problemáticas: "**¿puedo describir esta clase en una sola frase, sin usar la palabra 'y'?**" Dijimos que esa heurística anticipaba, sin nombrarlo, un principio formal.

Hoy le ponemos nombre, autor y reglas precisas: es el primero de los cinco principios **SOLID**.

CONCEPTO

Qué es SOLID

SOLID es un acrónimo que agrupa cinco principios de diseño orientado a objetos, propuestos y difundidos por **Robert C. Martin ("Uncle Bob")**, pensados para producir software más fácil de mantener y extender en el tiempo.

Letra	Principio	Cuándo lo vemos
S	Single Responsibility Principle	Hoy
O	Open/Closed Principle	Viernes
L	Liskov Substitution Principle	Viernes
I	Interface Segregation Principle	Semana 9
D	Dependency Inversion Principle	Semana 9

CONTRA EJEMPLO

La clase que hace todo: registro de usuarios

```
class GestorDeUsuario:
    def validar_datos(self, datos): ...
    def hashear_contrasena(self, contrasena): ...
    def guardar_en_base_de_datos(self, usuario): ...
    def enviar_correo_de_bienvenida(self, usuario): ...
    def registrar_en_log_de_auditoria(self, usuario): ...
```

Una sola clase mezcla validación de negocio, seguridad, persistencia, notificación y auditoría. **Cambiar el proveedor de correo obliga a tocar la misma clase que valida contraseñas.**

CONCEPTO

Definición formal de SRP

"A class should have only one reason to change."
— Robert C. Martin

Una clase debe tener una sola razón para cambiar. "Razón para cambiar" significa: un solo actor, área de negocio o motivo externo puede pedir que esa clase se modifique.

`GestorDeUsuario` tiene al menos cuatro razones para cambiar: reglas de validación, política de seguridad, esquema de base de datos y plantilla de correo — cuatro equipos o decisiones distintas pueden pedir un cambio ahí.

CONCEPTO

Error común: "un método" no es lo mismo que "una razón"

SRP **no dice** que una clase deba tener un solo método. Una clase con cinco métodos puede cumplir SRP perfectamente si los cinco giran alrededor de la misma responsabilidad; una clase con un solo método puede violarlo si ese método mezcla varias tareas no relacionadas.

Caso	¿Cumple SRP?
<code>CalculadoraDeImpuestos</code> con 5 métodos, todos sobre cálculo de impuestos	Sí — una sola razón para cambiar
<code>procesar()</code> , un único método que valida, calcula, guarda y notifica en 80 líneas	No — cuatro razones para cambiar, disfrazadas de un método

CONCEPTO

SRP formaliza lo que ya vimos como cohesión

La semana pasada dijimos: **alta cohesión** es que todos los métodos y atributos de una clase giren alrededor de un único propósito. SRP es esa misma idea, expresada con un criterio verificable: en vez de preguntar "¿está relacionado?", pregunta "**¿qué actor o motivo externo pediría este cambio?**"

Una clase de baja cohesión —la clase "todóloga" o **God Class** que vimos la semana pasada— es, dicho con el vocabulario de hoy, la violación más visible de SRP.

CONCEPTO

Cómo identificar "razones para cambiar"

Para cada método de la clase, pregúntense: **¿quién pediría este cambio?** Si las respuestas son personas o áreas distintas, la clase agrupa más de una responsabilidad.

- `validar_datos()` — lo pide el área de negocio, si cambian las reglas de registro.
- `hashear_contrasena()` — lo pide seguridad, si cambia el algoritmo de cifrado.
- `guardar_en_base_de_datos()` — lo pide el equipo de datos, si cambia el motor o el esquema.
- `enviar_correo_de_bienvenida()` — lo pide marketing/producto, si cambia la plantilla o el proveedor.

Cuatro actores distintos, cuatro razones para cambiar → cuatro clases candidatas.

CONCEPTO

Refactorizando hacia SRP

```
class ValidadorDeUsuario:
    def validar(self, datos): ...

class GestorDeSeguridad:
    def hashear_contrasena(self, contrasena): ...

class RepositorioDeUsuario:
    def guardar(self, usuario): ...

class NotificadorDeBienvenida:
    def enviar(self, usuario): ...
```

`GestorDeUsuario` se divide en cuatro clases, cada una con una sola razón para cambiar. **Cambiar el proveedor de correo ahora solo toca `NotificadorDeBienvenida`** — nada más se ve afectado.

CONCEPTO

Qué se gana aplicando SRP

- **Pruebas más simples:** cada clase se prueba de forma aislada, sin simular las otras cuatro responsabilidades.
- **Menos riesgo de regresión:** un cambio en la política de seguridad no puede romper accidentalmente el envío de correos.
- **Reutilización real:** `RepositorioDeUsuario` se puede usar en otro flujo (ej. actualización de perfil) sin arrastrar validación ni notificación.
- **Trabajo en equipo sin choques:** dos desarrolladores pueden tocar seguridad y notificaciones el mismo día sin pisarse en el mismo archivo.

CONTRA EJEMPLO

God Class: la violación de SRP llevada al extremo

Cuando una clase acumula responsabilidades durante meses sin que nadie la refactorice, se convierte en una **God Class** ("clase Dios"): decenas de métodos no relacionados, cientos o miles de líneas, y prácticamente todo el sistema depende de ella de alguna forma.

Es un **antipatrón** ampliamente documentado en la literatura de diseño de software: una clase así es difícil de probar, de entender y de modificar sin miedo — el destino más común de una clase que empezó violando SRP "solo un poco".



INTERACCIÓN

Encuentren las razones para cambiar

```
class ReporteDeVentas:
    def calcular_totales(self, ventas): ...
    def formatear_como_html(self, totales): ...
    def enviar_por_correo(self, reporte, destinatario): ...
    def guardar_copia_en_disco(self, reporte): ...
```

En el chat: identifiquen cuántas "razones para cambiar" distintas tiene esta clase, quién pediría cada una, y propongan un nombre para cada clase resultante del refactor.

5 minutos · respondan en el chat

TENSIÓN CONCEPTUAL

SRP no es "una clase por cada verbo"

Llevar SRP al extremo —una clase distinta por cada método, sin importar qué tan relacionados estén— produce el mismo problema que vimos con la sobre-fragmentación de la cohesión: decenas de clases diminutas que obligan a saltar entre demasiados archivos para entender un solo flujo.

El criterio no es "cuántas clases", sino "cuántos actores distintos pueden pedir un cambio en esta clase". Si la respuesta sigue siendo uno, agrupar esos métodos en la misma clase no viola SRP.

CONCEPTO · LO QUE VIENE

SRP es la base de los otros cuatro principios

Clases con una sola responsabilidad son más fáciles de **extender sin modificar** (Open/Closed, el viernes), más fáciles de **sustituir de forma segura** (Liskov, también el viernes) y más fáciles de conectar a través de **contratos pequeños** (Interface Segregation, semana 9).

SOLID no son cinco reglas sueltas — SRP ordena el terreno sobre el que se construyen las otras cuatro.

SÍNTESIS

Ideas clave de hoy

1. **SRP**: una clase debe tener una sola razón para cambiar (Robert C. Martin).
2. "Razón para cambiar" es un actor o motivo de negocio distinto — **no** es el número de métodos.
3. SRP formaliza con un criterio verificable lo que la semana pasada llamamos alta cohesión.
4. Una **God Class** es el destino de una clase que acumuló violaciones de SRP sin refactorizar.
5. Aplicado al extremo, SRP produce sobre-fragmentación — el criterio es "actores distintos", no "clases por verbo".



CIERRE

Para la próxima sesión

Revisen una clase del proyecto de Fábrica Escuela: para cada método, anoten quién pediría ese cambio. Si aparece más de un actor, esa clase es candidata a refactor con SRP — tráiganla identificada.

El miércoles es CoWorking — Review Sprint 1, con la Entrega 1 de Fábrica Escuela (15%). El viernes cerramos la semana con los siguientes dos principios SOLID: **Open/Closed** y **Liskov Substitution**.

